

Dynamic Function Exchange Guide for Versal

Vivado 2023.2

Table of content

Introduction	3
Required Prerequisites	4
DFX on VPK120 Tutorial	5
Step 1: Create a new Vivado Project	5
Step 2: Create Hierarchy Levels in the Block Design	6
Step 3: Create A Block Design Container (BDC)	7
Step 4: Enable Dynamic Function eXchange	10
Step 5: Add a New Reconfigurable Module	11
Step 6: Confirm Apertures for All Reconfigurable Modules	15
Step 7: Create a Wrapper and Generate the Targets for the Top BD	16
Step 8: Use the DFX Wizard to Define Configurations	17
Step 9: Add Design Constraints for the Reconfigurable Partition	19
Step 10: Implement the Configurations and Generate Bitstreams and XSAs.	20
Step 11: Creating a Petalinux BSP Project.	21
Step 12: Using the DFX Design on Versal	25
References	26

Document version manager

Date	Version	Description	Author
6.2026	1.0	Initial Release	Shay Saraf

Introduction

This guide provides instructions and best practices for implementing Dynamic Function Exchange (DFX) on the Versal architecture using the VPK120 evaluation board, the Ubuntu operating system, and Vivado 2023.2. It is authored by Shay Saraf and is intended to assist engineers in navigating the specific workflows and design considerations required for DFX in Versal-based projects.

Dynamic Function eXchange (DFX) allows for the modification of a portion of the FPGA design at runtime without affecting the operation of the remaining logic. This is achieved through the following core components:

- **Static Region:** The portion of the design that remains operational and fixed while other parts are being reconfigured.
- **Reconfigurable Partition (RP):** A specific area on the FPGA allocated for dynamic logic, which can be swapped out during operation.
- **Reconfigurable Module (RM):** The specific hardware design implementation that is loaded into a Reconfigurable Partition.

Required Prerequisites

This guide is intended for hardware and firmware development engineers who have previous experience and background with the Vivado development environment and the Versal adaptive SoC architecture. To get the most out of this tutorial, the reader should be familiar with the following core concepts and tools:

Hardware & Software Environment

- **Vivado 2023.2:** The specific version of the AMD/Xilinx Vivado Design Suite used throughout this guide for block design composition, synthesis, physical implementation, and exporting hardware definitions.
- **VPK120 Evaluation Board:** The targeted hardware platform featuring a Versal Premium device. This board provides the physical environment to implement and validate the runtime reconfiguration.
- **PetaLinux:** The embedded Linux software development kit from AMD/Xilinx. A basic understanding of PetaLinux workflows is required to build a custom Board Support Package (BSP), manage the boot image, and handle device tree overlays for live hardware updating.

Core Architecture & Methodology Concept:

- **PL (Programmable Logic):** The FPGA fabric portion of the Versal adaptive SoC where custom digital logic circuits, Reconfigurable Partitions (RPs), and Reconfigurable Modules (RMs) are physically mapped and executed.
- **CIPS (Control, Interfaces, and Processing System):** The hard IP core block that integrates the Arm processing system, platform management controllers, and hardened memory interfaces within the Versal architecture, serving as the system's static anchor.
- **AXI Protocol and Network on Chip (NoC):** A solid understanding of AXI interconnect interfaces and the Programmable Network on Chip (NoC) architecture, which routes high-bandwidth communications and manages memory-mapped address spaces between the static region and dynamic partitions.
- **GPIO (General Purpose Input/Output):** A standard peripheral interface used here as an AXI-mapped register. In this tutorial, the AXI GPIO IP block acts as the hardware container within the dynamic region to read out fixed constant identification codes, providing immediate software feedback on whether a module swap succeeded.

DFX on VPK120 Tutorial

To provide clear context and streamline the learning process, this guide is structured as a hands-on tutorial on building a DFX project from scratch with one Reconfigurable Partition (1 RP) and two Reconfigurable Modules (2 RMs).

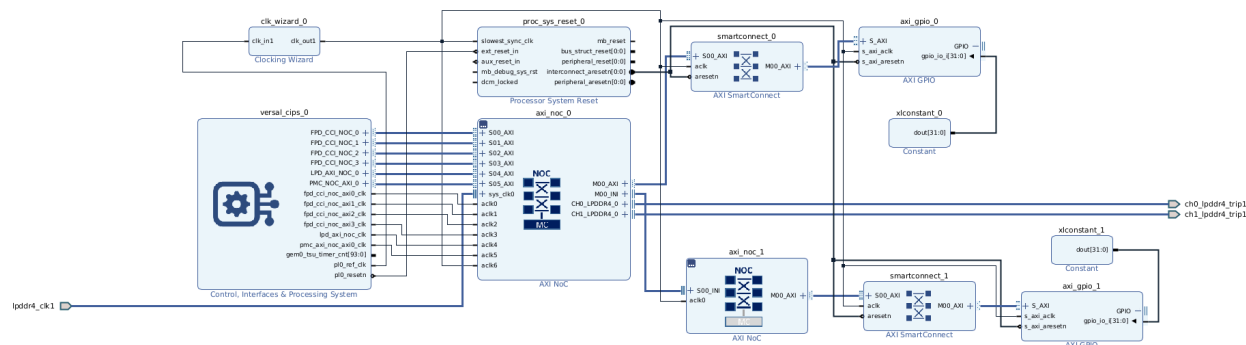
The essence of this demonstration and its logic is straightforward and highly focused: we will define a GPIO block inside the dynamic region where a different constant value is saved depending on which module is currently active:

- In the first module (rp1rm1), the GPIO will hold the constant **0xFEEDCODE**.
- In the second module (rp1rm2), the GPIO will hold the constant **0xFACEFEED**.

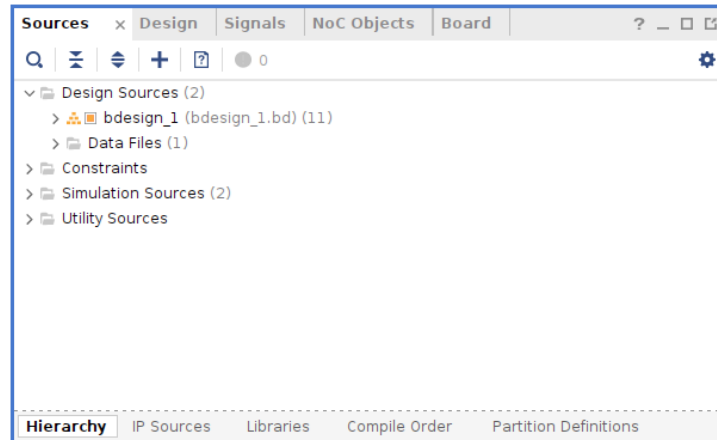
By swapping these hardware modules at runtime within the operating system (Ubuntu), we will be able to read the corresponding memory mapped addresses to verify that the values change successfully. This test will serve as definitive proof that the dynamic reconfiguration is functioning correctly. Understanding this basic structure is crucial for more advanced designs, as the principles and steps learned here can be easily extended to multi-RP multi-RM projects.

Step 1: Create a new Vivado Project

For this tutorial, please download the [DFX_Tutorial_vpk120.tcl](#) code from the linked repository. Open Vivado 2023.2 IDE and navigate from the TCL Console to the path you downloaded the .tcl file to. Then, run in the console: **source DFX_Tutorial_vpk120.tcl**. This will generate the project for the tutorial. The following figure shows the initial block design (bdesign_1.bd):



The following figure shows the initial design hierarchy:



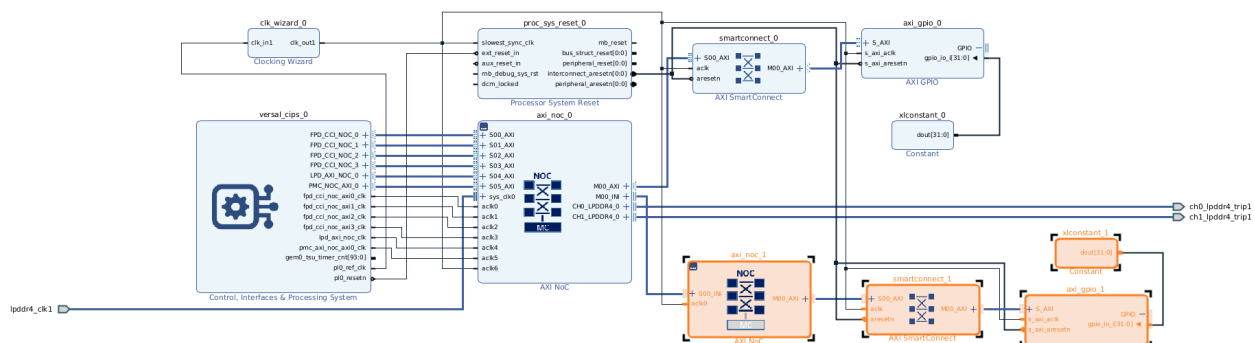
Step 2: Create Hierarchy Levels in the Block Design

In this section, you will split the design into two hierarchical instances:

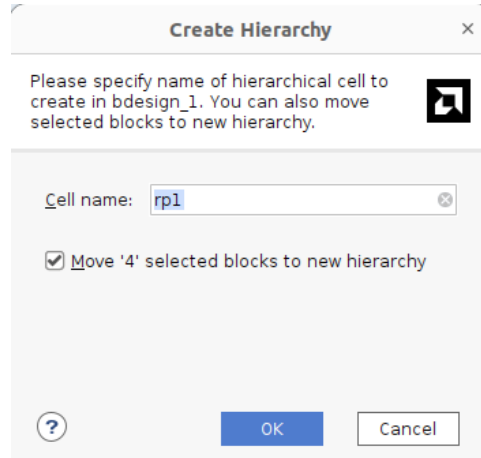
The Static Region: Creating a hierarchy here isn't strictly required, but it helps organize the design, focus on the dynamic area, and simplify floorplanning.

The Reconfigurable Partition: This hierarchy is mandatory, as it will be converted into a Block Design Container (BDC).

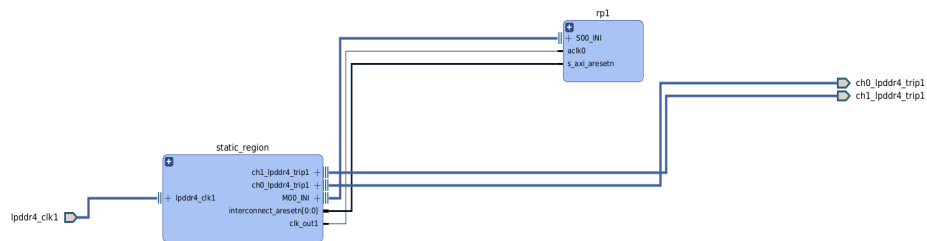
1. Ctrl-click to select these four instances in the block design: axi_noc_1, smartconnect_1, axi_gpio_1, and xlconstant_1. Then right-click on one of these four instances and **select Create Hierarchy**.



2. In the resulting dialog box, name the hierarchy **rp1** and click **OK**.



3. Right-click on the Versal CIPS instance and select **Create Hierarchy**. In the resulting dialog box, name the hierarchy **static_region** and click **OK**.
4. One by one, take all the remaining instances (other than rp1) and drag and drop them into the static_region level of hierarchy. The resulting block design can be cleaned up by running Regenerate Layout, and each level of the hierarchy can be collapsed. The resulting block design should look like this:

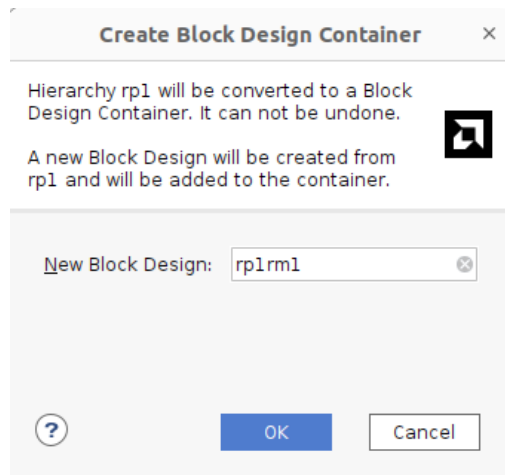


5. Click on **Validate Design** and then save the block design.

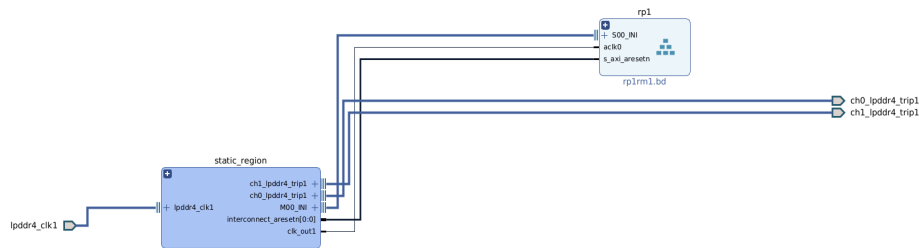
Step 3: Create A Block Design Container (BDC)

Now that levels of hierarchy are established, the rp1 instance can be converted to a block design container, which will represent the Reconfigurable Partition.

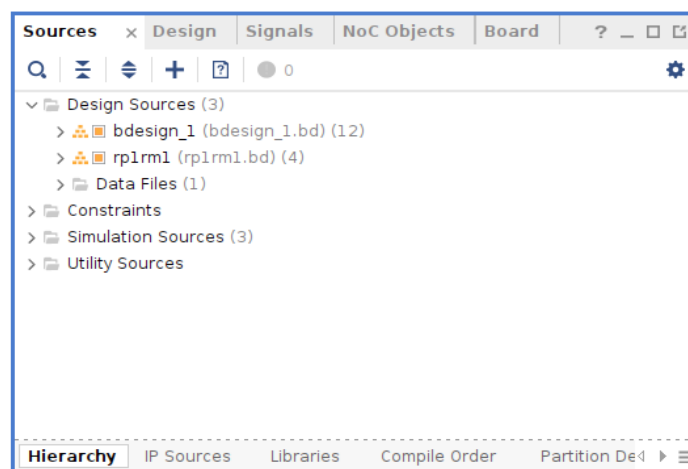
1. Right-click on the collapsed rp1 instance and select **Create Block Design Container**.
2. Name the container **rp1rm1** and click **OK**.



This will convert the hierarchical instance into a block design container. The level of the hierarchy is labeled `rp1rm1.bd` and the block contains an icon that looks like a pyramid of six rectangles.



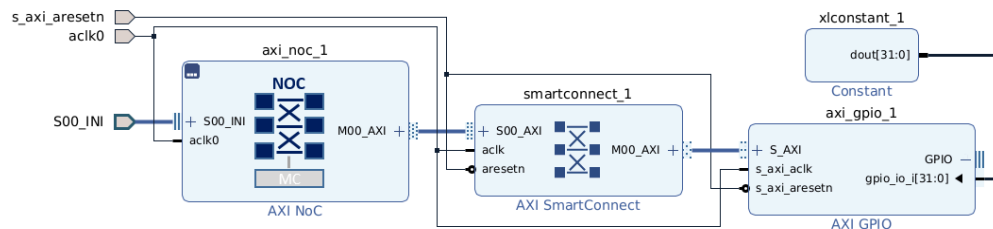
In the Sources window, you will see a new block design has been added to the project.



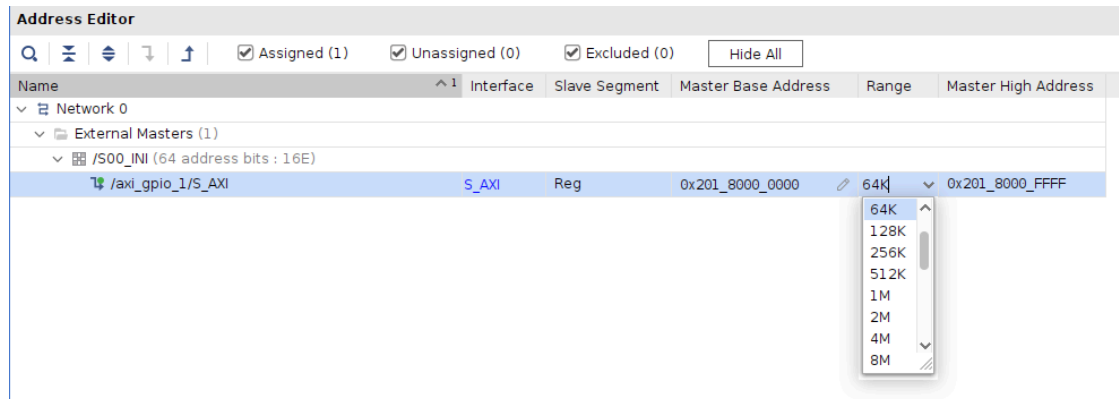
This action creates a new block design for the `rp1` submodule.

Because the rp1 instance within bdesign_1 is a read-only copy, you cannot edit it directly from that level. To make changes, you must open the source rp1rm1.bd block design from the Sources window.

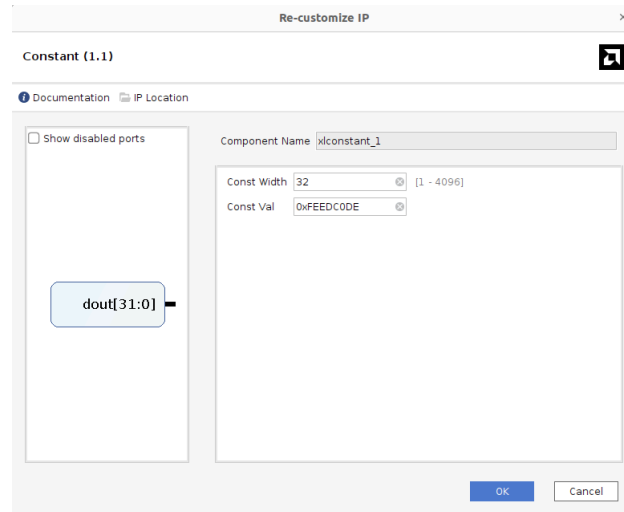
3. Double click on the rp1rm1 block design in the Sources window to open the block design.



4. Select the **Address Editor** window and ensure the range for /rp1/axi_gpio_1/S_AXI is **64K**.



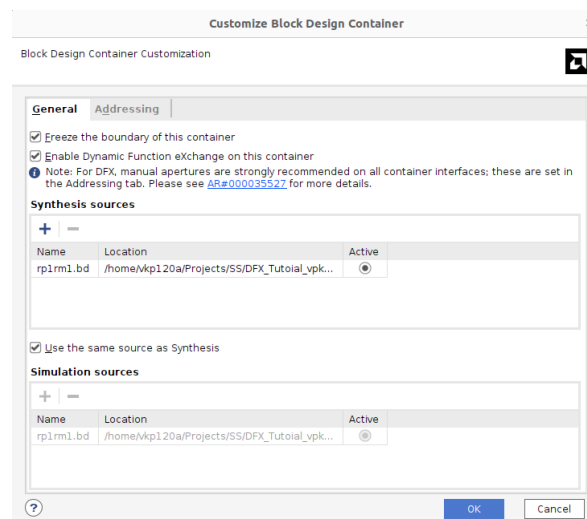
5. This block design is going to be our first RM. The constant that is saved in its GPIO is 0xFEEDCODE as can be seen by double clicking the xlconstant_1 block in this design.



Step 4: Enable Dynamic Function eXchange

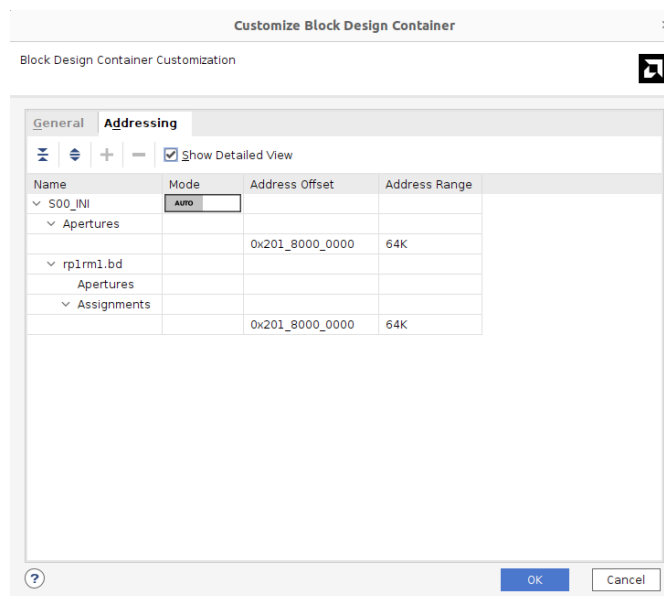
In this section, you will enable the DFX capabilities and add new Reconfigurable Modules in the rp1 block design container.

1. Select **Tools > Enable Dynamic Function eXchange** to expose DFX features within the Vivado IDE. Select the **Convert** option in the dialog box that opens. Once this step has been run, you will see new menu items appear, most notably the **Dynamic Function eXchange Wizard** in the Flow Navigator and under the Tools menu.
2. In the bdesign_1 diagram, double click on the **rp1** instance to edit the block design container.
3. Under the General tab, check both the **Enable Dynamic Function eXchange on this container** and the **Freeze the boundary of this container** options.

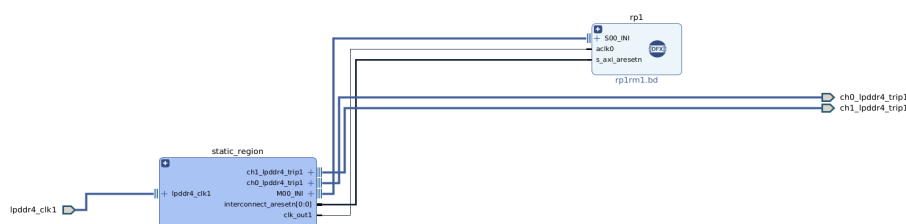


Checking the Enable Dynamic Function eXchange on this container option defines the rp1 instance to be a Reconfigurable Partition (RP). Freezing the boundary of this container prevents parameter propagation across the boundary interface.

- Click the Addressing tab to see the aperture for this block design container. The **Address Offset** is 0x201_8000_0000 and the **Address Range** is 64K, matching the information supplied in rp1rm1. Check the **Show Detailed View** to see that the aperture for rp1rm1 matches the general aperture for rp1 overall. No changes are necessary at this point. This tab will be revisited later.



- Click **OK** to save the changes and return to the bdesign_1 diagram. You will see that the icon on the rp1 block design container has changed to show a “DFX” label.

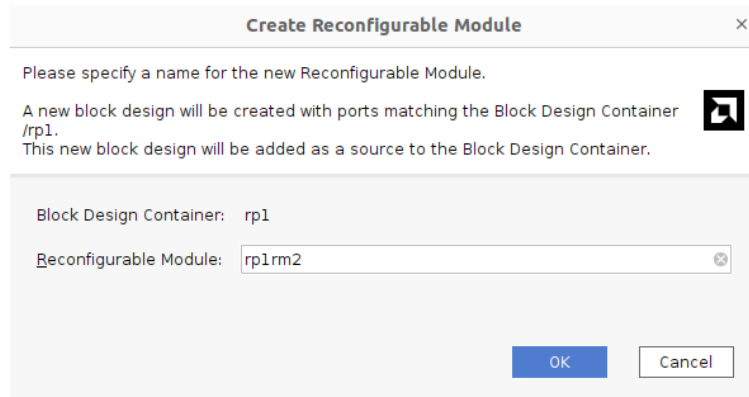


- Click **Validate Design** then **Save** to save the design.

Step 5: Add a New Reconfigurable Module

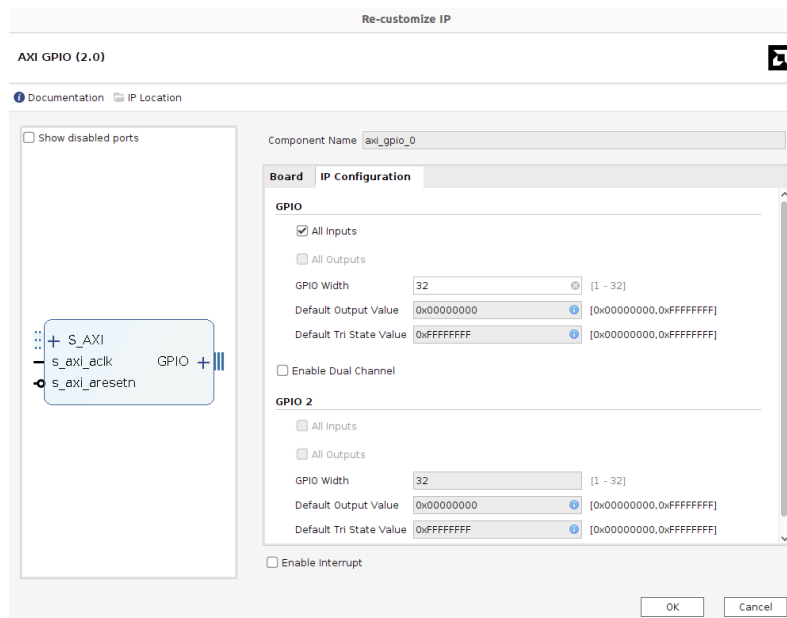
Dynamic Function eXchange requires multiple Reconfigurable Modules (RMs) to swap between. The next step is to create a new RM for your existing RP.

1. Right-click on the rp1 instance and select **Create Reconfigurable Module**. In the dialog box that opens, give the RM a name of **rp1rm2** and click **OK**.

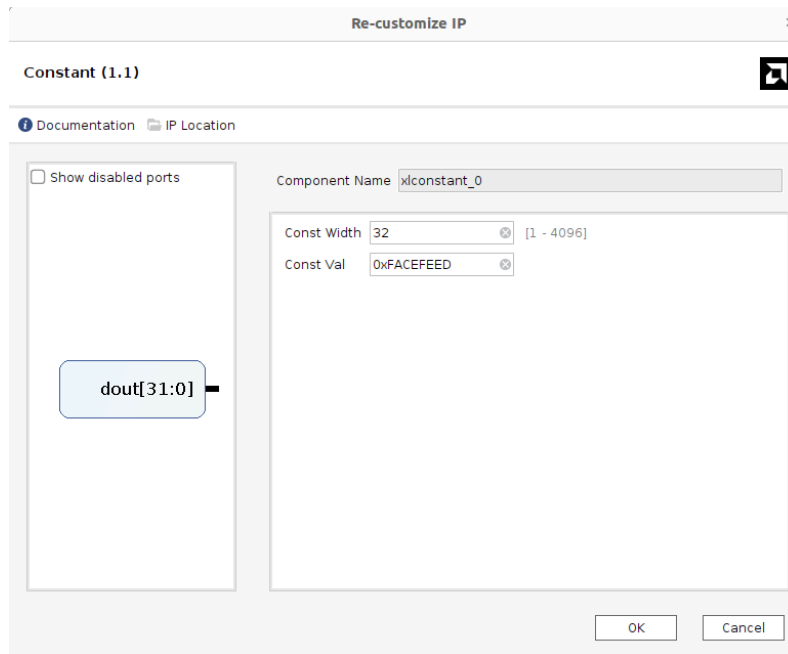


A new block design is created and opened. The diagram consists of three input pins, which are the same port list as the first RM for the rp1 partition. The port list for each RM for a given RP must be identical, even if not all of the ports are used by each RM. Note that in the log (and script) the `create_bd_design` command uses the `-boundary_from_container` option, copying the explicit port list from the block design container.

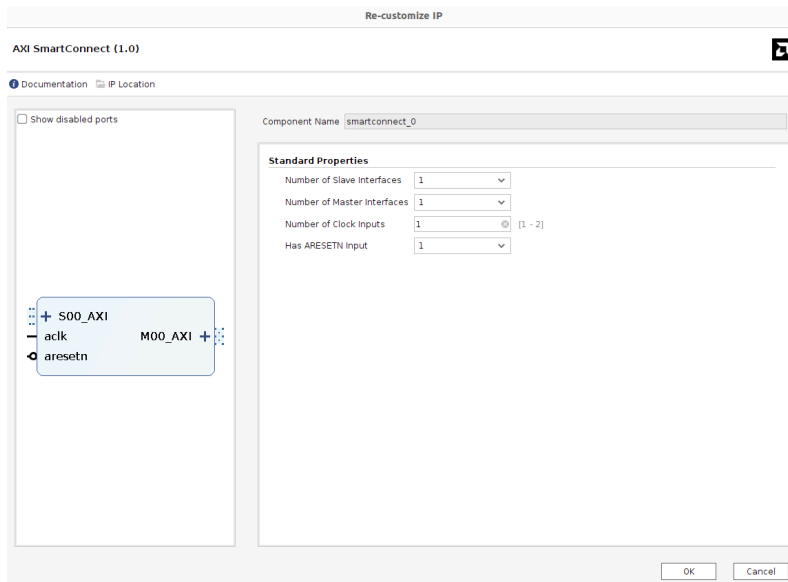
2. Add a new IP to the canvas by clicking the + icon and using the search field to find the AXI GPIO IP. Add it to the canvas, then double-click to customize. Check the **All Inputs** box for GPIO, ensure the **GPIO Width** is set to **32**, then **OK** to return to the canvas.



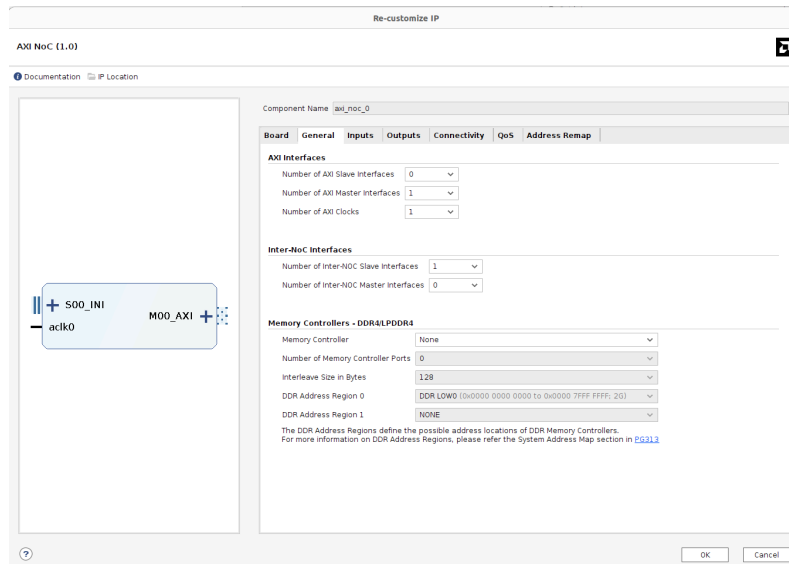
- Click the + again and use the search field to add a Constant IP to the canvas. Double-click to customize. Change the **Const Width** to 32 and **Const Val** to 0xFACEFEED (different from the constant value in rp1rm1). Click **OK** to accept the edits.



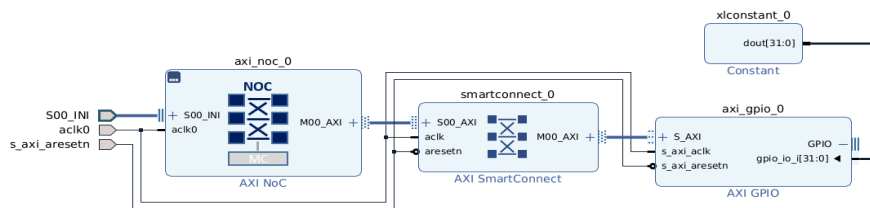
- Click the + again and use the search field to add an AXI SmartConnect IP to the canvas. Double-click to customize. Change the **Number of Slave Interfaces** to 1 so that all values are now 1. Click **OK** to accept the edits.



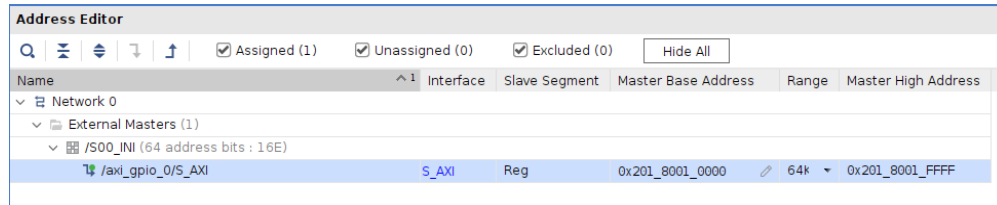
- Click the + one last time and use the search field to add an AXI NoC IP to the canvas. Double-click to customize. Under the General tab, set the **Number of AXI Slave Interfaces** to 0 and the **Number of Inter-NOC Slave Interfaces** to 1. On the Connectivity tab, check the single box to connect M00_AXI to S00_INI. Click **OK** to accept the edits.



- Connect the pins to create the diagram as shown in the following figure. Note: You will need to expand the GPIO port to expose the 32-bit input bus to match the type of the Constant dout bus.



- Change to the Address Editor window and note that no addresses have been assigned. Right-click on the row for /axi_gpio_0/S_AXI and select Assign. This sets a 64K range starting at address 0x0000_0202_0000_0000. Modify the Master Base Address so it starts at 0x0201_8001_0000 then keep the Range at 64K.



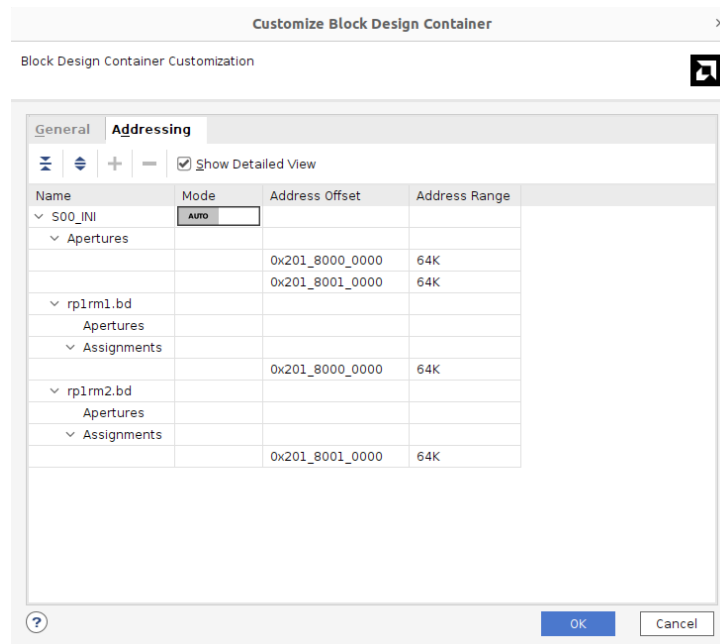
- Back in the block design, click **Validate Design** and then **Save** to save the rp1rm2 block design. In this simple design, there are only two differences between rp1rm1 and rp1rm2:

- The S_AXI base addresses are different.
- The constant values that can be read via GPIO are different.

This first difference will be used to show that device tree overlays must be created and managed for designs that may have different requirements between Reconfigurable Modules. The second difference will be used to confirm that dynamic reconfiguration in hardware has been done successfully.

Step 6: Confirm Apertures for All Reconfigurable Modules

- In the top-level block design, double click on the rp1 block design container. Switch to the **Addressing** tab and click the **Show Detailed View** checkbox.



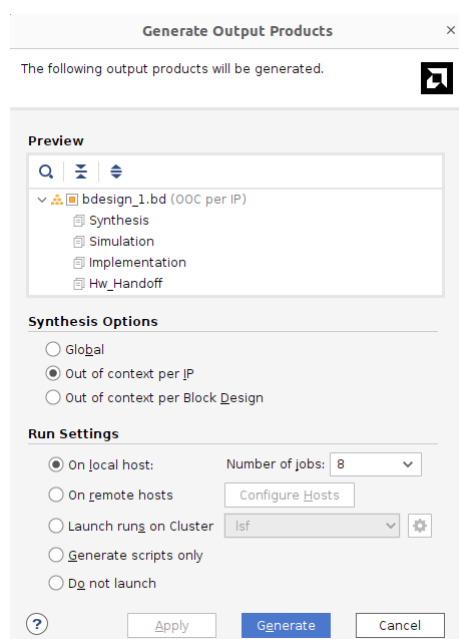
You can see that the overall aperture for rp1 for the S00_INI port starts at address 0x201_8000_0000 and has an overall range of 128K. This is automatically calculated by collecting address information from each design source in the block design container and summarizing each module's requirements.

2. Click **OK** to return to the top-level block design.
3. **Validate** and **save** bdesign_1.bd.

Step 7: Create a Wrapper and Generate the Targets for the Top BD

The final step before processing through synthesis and implementation is to create an HDL wrapper for the top-level block design, then generate targets for synthesis.

1. In the Sources window, right-click on bdesign_1.bd and select **Create HDL Wrapper**. Keep the Let **Vivado Manage** option selected and click **OK**. In the sources, bdesign_1_wrapper.v has been created and added to the project. This HDL file instantiates the bdesign_1 block design.
2. In the Flow Navigator, click the **Generate Block Design** command under the IP INTEGRATOR header. In the resulting dialog box, keep the Out of context per IP option selected, then click Generate.



This action generates synthesizable output products and creates out-of-context (OOC) synthesis runs for each IP in bdesign_1.

Under the Design Runs tab, you will see that:

- Static Region IP: All synthesis runs for IP within the static_region hierarchy (outside the rp1 container) are actively running.
- Reconfigurable IP: Runs for the rp1rm1 and rp1rm2 sources have been created but are not yet running; these will be triggered later.

Step 8: Use the DFX Wizard to Define Configurations

The Dynamic Function eXchange (DFX) Wizard defines the relationships between the components of a DFX design.

In this flow:

- Reconfigurable Partition (RP): Represented by the Block Design Container (BDC).
- Reconfigurable Module (RM): Each individual source that can be loaded into the RP.

Within the wizard, you will define:

Configurations: A complete design image containing exactly one RM per RP.

Configuration Runs: The implementation pass (place and route) that generates a routed checkpoint for a specific configuration.

The wizard automatically establishes parent-child relationships and dependencies between these runs. This automates static design locking and pr_verify, ensuring Vivado only reruns necessary steps when sources change.

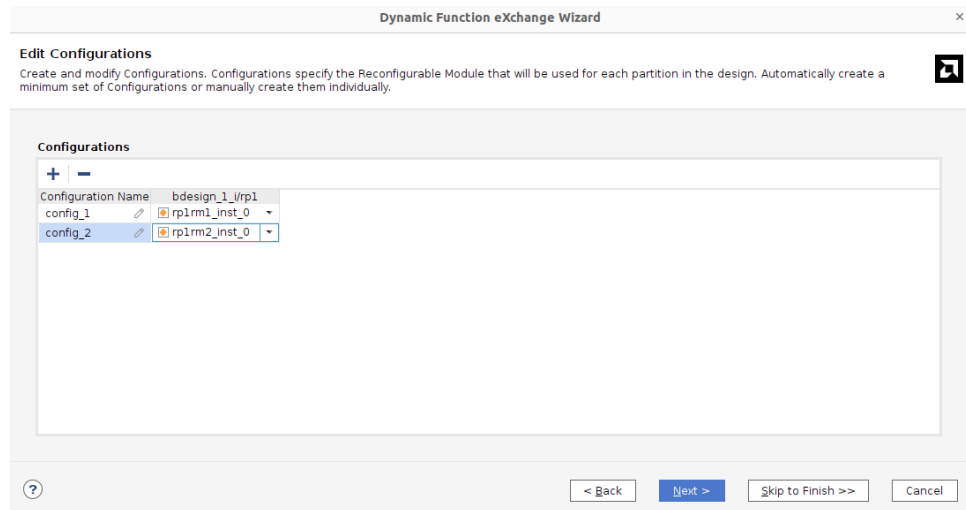
1. Open the DFX Wizard by clicking **Dynamic Function eXchange Wizard** in the Flow Navigator or by selecting that option under the Tools menu.



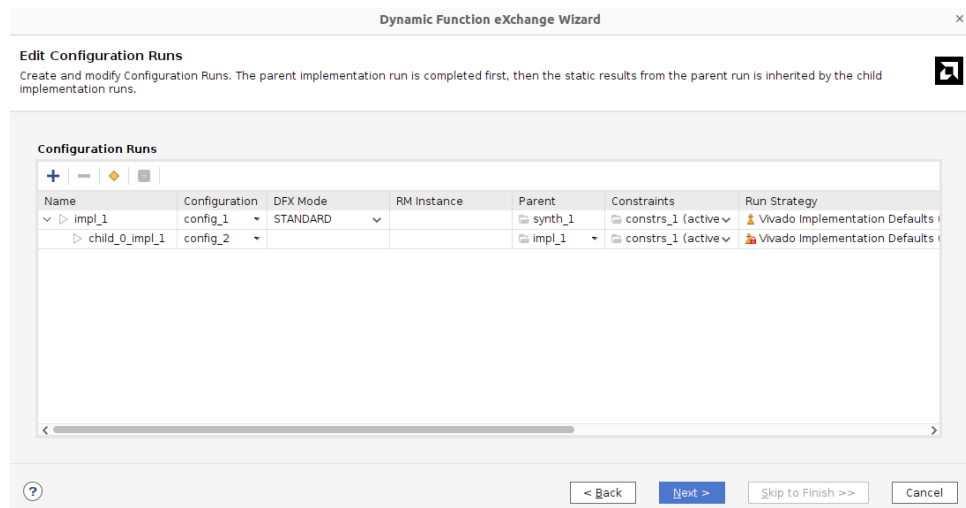
2. Click **Next**. In the Edit Reconfigurable Modules step, you will see the two RMs, rp1rm1_inst_0 and rp1rm2_inst_0, that you created within the rp1 block design container.
Important: Unlike within the RTL Project flow for Dynamic Function eXchange, the DFX Wizard is NOT the entry point for new Reconfigurable Modules. New RMs should be added from the canvas in the same way RM2 was added.
3. Click Next. In the Edit Configurations step, click the automatically create configurations link to generate two configurations.

Click the + button to generate configurations covering all RMs.

For multi-RP multi-RM DFX designs, we would need to cover all of the different configurations, so each RM will be used at least once.



- Click **Next**. In the Edit Configuration Runs step, click the **Standard DFX** link to create one run per configuration.



In the DFX flow, the Parent category establishes critical design consistency.

- Parent Run (impl_1): Starts with a standard synthesis and implementation run to lock down the static design.
- Child Run (child_0_impl_1): Must reference the parent run to inherit the locked static layout, ensuring the reconfigurable modules align perfectly.

In the Vivado interface, this relationship is visually represented by the indentation of the child run under its parent.

- Click **Next** then **Finish** to complete this section.

The DFX Wizard can be revisited any time to create new or modified configurations and configuration runs within the project.

In the Design Runs window, a new implementation run for child_0_impl_1 has been created. Like the view in the DFX Wizard, this new run is intended to show its dependency on the impl_1 run above it.

Tcl Console Messages Log Reports Design Runs x NoC QoS ? _ □ ▢					
Name	Configuration	DFX Mode	RM Instance	Constraints	Status
▼ > synth_1 (active)				constrs_1	Not started
> > impl_1 (active)	config_1	STANDARD		constrs_1	Not started
▼ Out-of-Context Module Runs					
> > rp1rm1_inst_0_synth_1				rp1rm1_inst_0	Not started
> > rp1rm2_inst_0_synth_1				rp1rm2_inst_0	Not started
> ✓ bdesign_1					Submodule Runs Complete

Step 9: Add Design Constraints for the Reconfigurable Partition

Establishing the Pblocks for the design is a necessary step in the floorplanning process.

- Press on the + in the **Sources** tab to create a new constraint file in your Vivado project called pblocks.xdc and paste the following Tcl commands into that file to define the boundaries for the Static Region and the first Reconfigurable Partition:

```
# Define the Static Region Pblock
create_pblock pblock_static_region
# Define and assign cells to the Reconfigurable Partition (rp1)
create_pblock pblock_rp1_1
add_cells_to_pblock [get_pblocks pblock_rp1_1] [get_cells -quiet [list bdesign_1_i/rp1]]
resize_pblock [get_pblocks pblock_rp1_1] -add
{CLOCKREGION_X3Y2:CLOCKREGION_X3Y2}
set_property SNAPPING_MODE ON [get_pblocks pblock_rp1_1]
```

After pasting, click on **Save**. For more advanced designs, we should carefully define independent Pblock for each Reconfigurable Partition. This isolation is crucial to prevent resource contention and to ensure that the placement of one module does not negatively impact the timing or routing of the other (we will see an example in part 2).

Step 10: Implement the Configurations and Generate Bitstreams and XSAs.

This final stage implements the DFX configurations, generates full and partial device bitstreams, and exports the required hardware platforms (.xsa) for PetaLinux deployment.

1. GUI Method (Manual Steps):

- **Launch Implementation:** Right-click child_0_impl_1 in the Design Runs window and select **Launch Runs**. This automatically executes the OOC synthesis runs, the parent implementation (impl_1 with locked static boundaries), and the dependent child implementation (child_0_impl_1) in the correct order.
- **Generate Bitstreams:** Shift-click to select both impl_1 and child_0_impl_1, right-click, and select **Generate Device Image** to create full and partial bitstreams.
- **Export Platforms:** Open impl_1, navigate to **File > Export > Export Hardware** (include the device image), and save it to the parent directory. Repeat this process for child_0_impl_1, saving it to the child directory.

2. Tcl Method (Automated Script)

The provided Tcl procedure (build_dfx_linux_deployment) automates the entire manual sequence into a single, optimized command.

Open the **TCL Console** in your project, copy the following tcl code, paste it in the console and run. This script will create a ./xsa folder in your home directory with all the required XSAs and the .pdi files of the parent and the child:

```
# XSA and pdi files exporter (run in tcl console after generating device image)
proc build_dfx_linux_deployment {} {
  puts " Starting Optimized DFX Build & Export Process (2 Runs)"
  puts "===== "
  set jobs 8
  set export_dir "./xsa"
  file mkdir $export_dir
  # 1. Run Synthesis
  puts "--> Launching Synthesis (synth_1)..."
  reset_run synth_1
  launch_runs synth_1 -jobs $jobs
  wait_on_run synth_1
  # 2. Run Implementation for Parent and Combined Child Run
  set impl_runs [list "impl_1" "child_0_impl_1"]
  foreach run $impl_runs {
    puts "--> Launching Implementation for: $run..."
    reset_run $run
    launch_runs $run -to_step write_device_image -jobs $jobs
    wait_on_run $run
    if {[get_property PROGRESS [get_runs $run]] != "100%"} {
      puts "ERROR: Run $run failed. Check Vivado logs."
      return
    }
  }
  # 3. Export Hardware Platforms (XSAs)
  puts "--> Exporting Hardware Platforms for PetaLinux..."
}
```

```

# --- From Parent Run (impl_1) ---
open_run impl_1
write_hw_platform -fixed -static -include_bit -force -file "$export_dir/static_region.xsa"
write_hw_platform -fixed -rp bdesign_1_i/rp1 -include_bit -force -file "$export_dir/rp1rm1.xsa"
close_design
# --- From Combined Child Run (child_0_impl_1) ---
open_run child_0_impl_1
write_hw_platform -fixed -rp bdesign_1_i/rp1 -include_bit -force -file "$export_dir/rp1rm2.xsa"
close_design
# 4. Collect PDIs from both runs
puts "--> Copying PDI files into XSA folder..."
# Parent PDI (Main Boot PDI)
set impl_1_dir [get_property DIRECTORY [get_runs impl_1]]
if {[file exists "$impl_1_dir/bdesign_1_wrapper.pdi"]} {
    file copy -force "$impl_1_dir/bdesign_1_wrapper.pdi" "$export_dir/bdesign_1_wrapper.pdi"
    puts " - Copied: impl_1 PDI -> $export_dir/bdesign_1_wrapper.pdi"
}
# Child 0 PDI (Contains the partials for your alternative modules)
set child_0_dir [get_property DIRECTORY [get_runs child_0_impl_1]]
if {[file exists "$child_0_dir/bdesign_1_wrapper.pdi"]} {
    file copy -force "$child_0_dir/bdesign_1_wrapper.pdi" "$export_dir/child_0_impl_1_bdesign_1_wrapper.pdi"
    puts " - Copied: child_0 PDI -> $export_dir/child_0_impl_1_bdesign_1_wrapper.pdi"
}
puts "===== "
puts " Build Complete! 3 XSAs and 2 PDIs are ready in '$export_dir'."
}
# Execute the procedure
build_dfx_linux_deployment

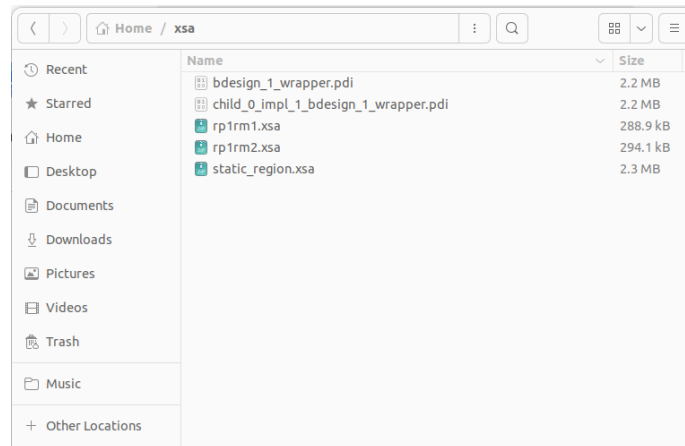
```

Step 11: Creating a Petalinux BSP Project.

This section helps to build and boot the DFX design on a Versal platform with a Petalinux BSP project, and flash its image to a SD card.

For this section, you will need to download the .bsp file of your Versal board and the Petalinux tool package version 2023.2 (can be found [here](#)). Also, you will need a SD card to flash the image on.

After generating the required XSAs and .pdi files, locate them on your PC in the ./xsa/ folder.



Now, open your PC Terminal follow the instructions:

1. Source the PetaLinux tool.

```
source <petalinux 2023.2 path>/settings.sh
```

2. Create a Versal template project or bsp project.

```
petalinux-create -t project -s <bsp path> -n versal-dfx
```

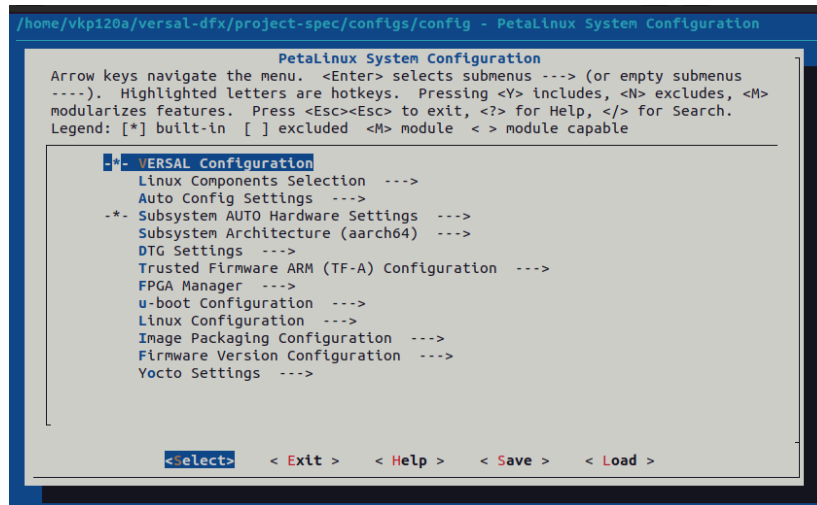
3. Go to the project.

```
cd versal-dfx
```

4. Configure the project with static.xsa.

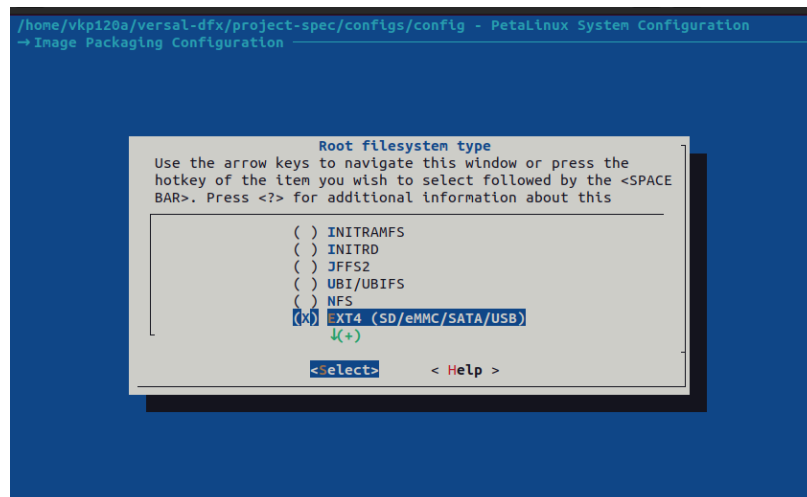
```
petalinux-config --get-hw-description <xsa path>/static_region.xsa
```

This command will open the system configuration GUI which will look like this:



In here, you will need to do two things:

- Using the keyboard arrows, move down until **"FPGA Manager"** and press Enter. Then, press "y" on your keyboard and make sure that now there is a * in the **FPGA manager** line. Then using the right arrow move to **"Exit"** and return to the home screen like in the picture above.
- Using the keyboard arrows again, move down to **"Image Packaging Configuration"** and press Enter. Then, in the opened page, press Enter once again to open **Root filesystem type**. In the opened page, move down to **EXT4 (SD/eMMC/SATA/USB)** and press Enter. Make sure that the root filesystem type is now EXT4.



Now, using the right arrow, move to **Save** and press Enter three times to save the changes. Now you can move to **Exit** and press Enter to return to the home screen, and move to **Exit** again and press Enter to exit the system configuration GUI back to the Terminal.

5. Create the static application using the following command from static xsa.

```
petalinux-create -t apps --template dfx_dtg_versal_static -n static --enable --srcuri "<xsa
path>/static_region.xsa"
```

6. Create the partial application to configure the partial region using the rm xsa in the following command. You should point static pl app name as --static-pn command line option to define the relation between base and partial.

```
petalinux-create -t apps --template dfx_dtg_versal_partial -n rp1rm1 --enable --srcuri "<xsa
path>/rp1rm1.xsa" --static-pn static
```

```
petalinux-create -t apps --template dfx_dtg_versal_partial -n rp1rm2 --enable --srcuri "<xsa
path>/rp1rm2.xsa" --static-pn static
```

7. Build the Petalinux project.

```
petalinux-build
```

The command generates the rootfs containing both static and rprm dtbos, and respective pdi files.

8. After the Petalinux project is built, build each application as well.

```
petalinux-build -c static
```

```
petalinux-build -c rp1rm1
```

```
petalinux-build -c rp1rm2
```

9. Package the Image of the project and get its .wic file ready for flashing on a SD card.

```
petalinux-package --force --boot --format BIN --plm --psmfw --fpga --u-boot
```

```
petalinux-package --wic
```

Now, the .wic file is at ./versal-dfx/images/linux and is ready to be flashed.

10. Open “Disks” on your PC and restore the image of the SD card using the .wic you have just created.

Now your SD card is ready to be inserted into the Versal.

Step 12: Using the DFX Design on Versal

After inserting the SD card, turn on the Versal board and wait for it to upload.

1. Run this command to create the overlay of the static design (and the default RM as part of it):

```
sudo fpgautil -b /lib/firmware/xilinx/static/static.pdi -o /lib/firmware/xilinx/static/static.dtbo
```

2. Now, lets create the overlays of both RMs:

```
sudo fpgautil -b /lib/firmware/xilinx/static/rp1rm1/rp1rm1.pdi -o  
/lib/firmware/xilinx/static/rp1rm1/rp1rm1.dtbo -f Partial -n rp1rm1
```

```
sudo fpgautil -b /lib/firmware/xilinx/static/rp1rm2/rp1rm2.pdi -o  
/lib/firmware/xilinx/static/rp1rm2/rp1rm2.dtbo -f Partial -n rp1rm2
```

The “-f Partial -n <rm app>” flags notifies that this overlay is part of a reconfigurable partition, so it can be switched by another RM..

3. Now its a good time to remember that in rp1rm1 we saved the constant 0xFEEDC0DE at address 0x201_8000_0000, and in rp1rm2 we saved the constant 0xFACEFEED at address 0x201_8001_0000, and that in the static region there was another GPIO that saved the constant 0xDFDFDFDF at address 0x201_0000_0000.

For checking if our DFX works, we need to be able to read from 0x201_0000_0000 no matter in which RM we use at the moment, be able to read from 0x201_8000_0000 only when we use rp1rm1 and be able to read from 0x201_8001_0000 only when we use rp1rm2.

For switching between RMs, we will use the following command:

For switching to rp1rm1:

```
sudo fpgautil -b /lib/firmware/xilinx/static/rp1rm1/rp1rm1.pdi
```

For switching to rp1rm2:

```
sudo fpgautil -b /lib/firmware/xilinx/static/rp1rm2/rp1rm2.pdi
```

4. When using rp1rm1, we need to be able to read from 0x201_0000_0000 and from 0x201_8000_0000 (if you will try to read from 0x201_8001_0000 the board will crash because you tried to read from an address that he doesn't know). Reading can be done as so:

```
sudo devmem 0x201_0000_0000 # this reads the gpio in the static part  
sudo devmem 0x201_8000_0000 # this reads the gpio in rp1rm1
```

5. When using rp1rm2, we need to be able to read from 0x201_0000_0000 and from 0x201_8001_0000 (if you will try to read from 0x201_8000_0000 the board will crash because you tried to read from an address that he doesn't know). Reading can be done as so:

```
sudo devmem 0x201_0000_0000 # this reads the gpio in the static part  
sudo devmem 0x201_8001_0000 # this reads the gpio in rp1rm1
```

This concludes the DFX tutorial for Versal.

References

1. [Vivado Design Suite Tutorial: Dynamic Function eXchange \(UG947\)](#)
2. [PetaLinux Tools Documentation: Reference Guide \(UG1144\)](#)